Phase 2 Evaluation Prep: Stack Overflow Developer Q&A Lakehouse

Team: Muhammad Sufyan (24L-2601), Muaaz Fahad (24L-2563). **Course:** Data Analysis and Visualization.

This guide prepares you for the Phase 2 evaluation. It follows the rubric line by line. For each line it explains what the examiner wants, what our code does, where that code is, and how to show it live in Databricks. After that come a file-by-file tour of the code, a step-by-step live demo script, and a bank of likely questions with answers.

Remember the rubric's last rule: *"Grading will be done on what your pipeline does, not what the README says it does."* So for every point, be ready to **show it running**, not only describe it.

---

1. The 60-second pitch

Learn this paragraph so you can say it without notes.

"We built the Bronze and Silver layers of a Medallion lakehouse on Databricks Free Edition. The data is Stack Overflow questions and answers for 18 data and analytics tags, pulled from the Stack Exchange API v2.3. The API extracts land as JSON-lines files in a Unity Catalog volume, one folder per batch. **Raw-to-Bronze** reads every file with an explicit `StructType` schema, records which batch and file each row came from, sends bad records to a quarantine table and handles new columns with `mergeSchema`. **Bronze-to-Silver** converts epoch numbers to timestamps, removes or hashes personal data, applies data-quality rules, removes duplicates and upserts every Silver table with `MERGE INTO`, so running the same input twice changes nothing. Both steps take parameters (mode, batch ids, date range, folder path), so any past period can be re-run without editing code. Every file or table processed writes a row to `so_ops.pipeline_execution_logs` with the layer, parameter, start and end time, status and row counts."

---

2. The project on one page

Architecture

```
Stack Exchange API
      |  ingestion/*.py (full_load.py, incremental_load.py)
      v
Landing volume   /Volumes/workspace/so_raw/landing/{full|incremental}/<batch_id>/
      |          questions.jsonl, answers.jsonl, deleted_questions.jsonl,
      |          question_snapshots.jsonl, _manifest.json
      |  notebooks/01_raw_to_bronze  ->  src/so_lakehouse/bronze.py
      v
Bronze  (workspace.so_bronze)  questions, answers, deleted_questions,
      |                         question_snapshots, quarantine_records
      |  notebooks/02_bronze_to_silver  ->  src/so_lakehouse/silver.py
      v
Silver  (workspace.so_silver)  questions, question_tags, answers, users,
                                question_snapshots, post_text_masked, dq_quarantine

Ops     (workspace.so_ops)     pipeline_execution_logs, batch_registry, schema_drift_events
```

Layer	What it holds	How it is written
Landing	Raw API output, untouched. One folder per batch.	`ingestion/*.py` or `00_setup` (copies the demo batches)
Bronze	Every raw record typed by the contract, one row **per record per batch**, plus lineage columns	Overwrite of that batch's rows only (`replaceWhere`)
Silver	One current, cleaned, PII-safe row per entity	`MERGE INTO`
Ops	Logs, batch registry, drift events	Append (logs) and `MERGE INTO` (registry, drift)

The data we ran with

The repo ships five real batches in `data/demo_landing/`, and the drift demo adds a sixth synthetic one.

Batch id	Type	Extracted (batch date)	Contents
`full_2025-04-01_2025-07-01`	full	2026-09-25	2,738 questions, 3,384 answers

Batch id	Type	Extracted (batch date)	Contents
`incr_20260925T155859Z`	incremental	2026-09-25	35 questions, 59 answers, 0 deletions, 100 snapshots
`full_2025-01-01_2025-04-01`	full	2026-10-07	4,548 questions, 5,528 answers
`incr_20261007T155914Z`	incremental	2026-10-07	305 questions, 646 answers, 1 deletion, 7,285 snapshots
`incr_20261007T160033Z`	incremental	2026-10-07	51 questions, 84 answers (smallest, best for a quick live demo)
`incr_20261008T000000Z`	synthetic drift demo	2026-10-08	5 lines: 2 valid, 3 broken on purpose

The full-load folder names describe the **content** window (questions created January to March 2025, for example). The batch date is when we **extracted** it. That is why the January to March batch has a later batch date than the April to June one.

Batch id is the folder name. **Batch date** is the UTC date the batch was extracted, read from `_manifest.json` (`extracted_at_utc`). Date-range backfills filter on this extraction date, not on when the questions were posted.

Numbers to remember (from Databricks job run `814281740296490`, 11 Oct 2026)

Fact	Value
Tables created	15 (5 Bronze, 7 Silver, 3 Ops)
Bronze rows after the first load	24,764 from 16 files
Silver rows after the first load	74,352 inserted, 7,285 updated (the updates are metrics from snapshots)
Re-run of the same input (backfill of all batches into Silver)	90,750 source rows, **0 inserted, 0 updated**
Re-run of the same input into Bronze	24,766 rows replaced, not duplicated
Duplicate primary keys in Silver	**0** in every table
Drift demo	5 lines read, 2 loaded, 3 quarantined, status SUCCESS
Log rows written in the whole run	67
Silver after the run	questions 7,620; answers 9,611; question_tags 24,185; users 8,332; question_snapshots 7,385; post_text_masked 17,231

All of this is saved, with outputs, in `evidence/`. If the live demo fails for some reason (internet, quota), open those notebooks on GitHub instead.

3. The rubric, line by line

Each subsection has the same four parts: what the examiner wants, what we did, where it is in the code, and how to show it.

3.1 Infrastructure and GitHub

Rubric: Use Databricks (Free Edition is fine). All PySpark code in GitHub with a sensible folder structure.

What we did:

- Databricks Free Edition, serverless compute, Unity Catalog, catalog `workspace`.
- Repo `Suf-7/stackoverflow-lakehouse`, with a clear split between reusable code and thin notebooks:

Folder	Role
`src/so_lakehouse/`	The pipeline as a Python package. All the logic lives here.
`notebooks/`	Databricks notebooks `00` to `04` and `99`. They only define widgets and call the package.
`ingestion/`	API extraction scripts that write landing batches
`config/pipeline_config.json`	Catalog and schema names, tracked tags, secret names
`jobs/`	Databricks job definition (two tasks, shared parameters)
`data/demo_landing/`	Five real batches used for the demo and tests
`tests/`	14 end-to-end tests on local Spark + Delta
`evidence/`	The Databricks run saved as Jupyter notebooks with outputs

Folder	Role
`docs/`	Proposal, data dictionary, learning guide, this guide

Why this structure (likely question): notebooks are hard to test and review. Putting logic in a package means the same code runs in Databricks, in a scheduled job, from the CLI (`so_lakehouse/cli.py`) and in the local tests. `notebooks/_bootstrap.py` adds `src/` to the Python path so the notebooks can import it.

Show it: the GitHub repo page, then the workspace folder in Databricks.

3.2 Data models and README

Rubric: Document every Bronze and Silver table in the README: column names, data types and primary key.

What we did: the README's "Data model" section lists every table with every column, type, nullability, comment and primary key. `docs/data_dictionary.md` has the same content.

The important design point: the documentation is **generated from the code**. `scripts/generate_data_dictionary.py` reads the `StructType` objects in `schemas.py` and writes the tables between the `DATA-DICTIONARY` markers in the README. So the README cannot disagree with the real schemas. The same `TABLES` registry in `schemas.py` also creates the tables in Databricks (`ddl.py`), with column comments, a table comment and a Unity Catalog `PRIMARY KEY` constraint.

Primary keys:

Table	Primary key	Why
`bronze.questions`	`question_id, _batch_id`	Bronze keeps history: the same question can appear in many batches
`bronze.answers`	`answer_id, _batch_id`	Same
`bronze.deleted_questions`	`question_id, _batch_id`	Same
`bronze.question_snapshots`	`question_id, snapshot_at, _batch_id`	One metric reading per question per time per batch
`bronze.quarantine_records`	`quarantine_id`	SHA-256 of entity, batch, file and raw line
`silver.questions`	`question_id`	One current row per question
`silver.answers`	`answer_id`	One current row per answer
`silver.question_tags`	`question_id, tag`	Bridge table, one row per pair
`silver.users`	`user_key`	Hashed user id
`silver.question_snapshots`	`question_id, snapshot_at`	Metric history
`silver.post_text_masked`	`post_type, post_id`	Questions and answers share this table
`silver.dq_quarantine`	`dq_id`	SHA-256 of entity, key, rule and batch

Show it: README on GitHub, then in Databricks **Catalog -> workspace -> so_silver -> questions** shows the columns, comments and primary key. Or run section 2 of `03_audit_and_validation` (`DESCRIBE TABLE` for all 12 tables and the primary keys from `information_schema`).

Likely question: *"Are Unity Catalog primary keys enforced?"* No. They are informational. Uniqueness is guaranteed by our code: Silver de-duplicates the source to one row per key before every `MERGE`, and `MERGE` never inserts a key that already exists. Key columns are also declared `NOT NULL`, which Delta does enforce.

3.3 Schemas and data types

Rubric: Define the schema with `StructType`/`StructField` before reading any file. Don't use `inferSchema`. Cast every field to its proper type when it moves to Silver.

What we did:

- `schemas.py` defines a contract for every raw file: `QUESTION_RAW`, `ANSWER_RAW`, `DELETED_RAW`, `SNAPSHOT_RAW` (with nested `OWNER` and `MIGRATED_FROM` structs). The helper `col()` builds a `StructField` and stores a comment in its metadata.
- `RAW_CONTRACTS` maps each entity to its file name, its contract and its primary key.

- `bronze.load_entity_file` reads each file as plain text lines, then parses every line with `F.from_json(value, contract, {"mode": "PERMISSIVE", "columnNameOfCorruptRecord": "_corrupt_record"})`. The schema is fixed before the file is touched.
- `inferSchema` appears **zero** times in `src/`. Notebook 03 counts it, and the test `test_no_schema_inference_in_source` fails if anyone adds it.

Why read as text first and then `from_json` (likely question): reading the line as text keeps the original line. We need it to (1) compute `_record_hash` (SHA-256 of the raw line), (2) store the untouched line in `quarantine_records.raw_record` for later replay, and (3) run our contract checker on it. Spark's schema-on-read silently drops unknown fields and turns wrong types into nulls, so without the original text we could not detect drift.

Casting into Silver (`silver.py`):

Raw (Bronze)	Silver	Code
`creation_date` bigint (epoch seconds)	`created_at` TIMESTAMP	`ts()` = `F.timestamp_seconds(col.cast("bigint"))`
`last_activity_date`, `last_edit_date`, `closed_date`	`*_at` TIMESTAMP	`ts()`
`title` with HTML entities (`&`, `'`)	decoded text	`_html_unescape()` UDF using `html.unescape`
`closed_reason` free text	`closed_reason_code` normalised, e.g. `needs_details_or_clarity`	`regexp_replace` + `lower`
`owner.user_id` bigint	`owner_user_key` string (salted SHA-256)	`user_key()`
`body` HTML	numeric features: `body_length`, `code_block_count`, `link_count`, ...	`occurrences()`
every column	exact contract type and order	`conform(df, SILVER_*)` casts every column to the Silver `StructType`

`conform()` is the guarantee: whatever the transformation produced, the DataFrame is cast to the Silver contract column by column before it reaches `MERGE`.

Show it: section 1 of `03_audit_and_validation` prints every contract as a schema tree, prints the reading code and the `inferSchema` count. Section 2 shows Bronze dates as `bigint` and Silver dates as `timestamp`.

3.4 `load_timestamp`

Rubric: Every row in every Bronze and Silver table needs a `load_timestamp`.

What we did: `load_timestamp` is a column in all 15 tables (it is in `BRONZE_METADATA`, every Silver schema and every Ops schema). It is set with `F.current_timestamp()` when the row is written.

- **Bronze:** when the record was written to Bronze. A backfill of the batch writes the rows again, so they get a new `load_timestamp`.
- **Silver:** when the row was last inserted or updated. Silver also has `first_loaded_at`, which `MERGE` never overwrites, so you can see both "first arrived" and "last changed".
- On an idempotent re-run Silver rows are not updated, so `load_timestamp` stays the same. That is correct: nothing was loaded.

Show it: section 3 of `03_audit_and_validation`: rows, null count (0 everywhere), earliest and latest `load_timestamp` per table.

3.5 Bronze layer

Rubric: Load the raw data as it arrives and keep track of where each record came from (source, file or batch).

What we did: Bronze keeps the raw contract columns unchanged (dates still epoch, `owner` still nested, `body` still HTML) and adds lineage metadata to every row:

Column	Meaning
`_batch_id`	Landing folder name, e.g. `incr_20261007T155914Z`
`_load_type`	`full` or `incremental`
`_batch_date`	UTC extraction date (used by date-range backfills)
`_extracted_at`	Extraction time from `_manifest.json`
`_source_file`	Full path of the file, taken from Spark's `_metadata.file_path`

Column	Meaning
`_record_hash`	SHA-256 of the raw JSON line
`_rescued_data`	JSON of fields that are not in the contract (drift), else null
`load_timestamp`	When the row was written to Bronze

How a batch is written (`bronze.load_entity_file`):

```
good.write.format("delta").mode("overwrite") \
    .option("replaceWhere", "_batch_id = '<batch_id>'") \
    .saveAsTable(target)
```

`replaceWhere` means "overwrite only the rows matching this condition". Loading a batch a second time replaces that batch's rows in one atomic Delta commit. Other batches are untouched, and no duplicates appear. The log row records how many rows were replaced in `rows_deleted`.

Batch registry: after each batch, `ops.batch_registry` is upserted with `bronze_status` and `bronze_loaded_at`. In `incremental` mode a batch with `bronze_status = 'SUCCESS'` is skipped and logged as `SKIPPED`. A batch that failed before is retried automatically.

Show it: section 3 of notebook 03 (rows grouped by `_batch_id`, `_load_type`, `_extracted_at`, `_source_file`), or the SQL in section 5.4.

Likely question: *"Why not MERGE in Bronze too?"* Bronze is a history: one row per record per batch. There is nothing to "update". The unit of re-processing is a batch, so replacing exactly that batch's rows is simpler and cheaper than a `MERGE`, and it is still idempotent.

3.6 Silver layer and data quality

Rubric: Clean and validate the data, fix the types, and deal with duplicates and bad records.

What we did (`silver.py`), in the order the steps run (`STEPS` list):

Step	Target	What happens
`step_questions`	`silver.questions`	De-duplicate, cast, hash owner, derive features, DQ rules, `MERGE`
`step_question_tags`	`silver.question_tags`	Explode tags into pairs; insert new pairs, delete pairs removed by an edit
`step_answers`	`silver.answers`	Same pattern as questions
`step_users`	`silver.users`	Build pseudonymous users from question and answer owners
`step_snapshots`	`silver.question_snapshots`	Score and view history
`step_apply_snapshots`	`silver.questions`	Copy the newest snapshot metrics onto questions
`step_deletions` (x2)	`silver.questions`, `silver.answers`	Soft delete: `is_deleted = true`
`step_post_text`	`silver.post_text_masked`	Masked titles and bodies for text analysis

Duplicates. The same question can appear in several batches (a full load and later incremental loads). `latest_per_key()` keeps one row per key using a window function: newest `_extracted_at`, then newest `last_activity_date`, then `_record_hash` as a tie-break so the result is deterministic. `MERGE` requires unique source keys, so this step is also what makes `MERGE` safe.

Data-quality rules (`question_dq_rules()`, `answer_dq_rules()`):

Entity	Rule
questions	`created_at` not null; `last_activity_at >= created_at`; title not blank
answers	`question_id` not null; `created_at` not null; `last_activity_at >= created_at`

`split_dq()` tags each row with the first rule it fails. Rows that pass go to `MERGE`. Rows that fail go to `silver.dq_quarantine` with the rule name and the full payload as JSON, so nothing is silently lost.

Two levels of bad-record handling (say this clearly):

1. **Structural** problems (bad JSON, wrong type, no key) are caught in Bronze and go to `bronze.quarantine_records`.
2. **Business** problems (dates in the wrong order, blank title) are caught in Silver and go to `silver.dq_quarantine`.

PII (from Phase 1):

- **Dropped** before Silver: `owner.display_name`, `owner.profile_image`, `owner.link`, `owner.account_id`.
- **Pseudonymised:** `owner.user_id` becomes `sha2(salt + ":" + user_id, 256)`. The salt is in the Databricks secret `so_lakehouse/pii_salt`, never in code or output.
- **Masked** in `silver.post_text_masked`: emails become `[EMAIL]`, IP addresses `[IP]`, `password=...` style values `[SECRET]`, and database URLs `[CONNECTION]`.

Deletes and metrics:

- Deleted questions arrive as tombstones in `deleted_questions.jsonl`. Silver marks them `is_deleted = true` instead of removing them (soft delete), and answers of a deleted question are marked too.
- Votes and views do not change `last_activity_date`, so fresh metrics arrive in `question_snapshots.jsonl`. `step_apply_snapshots` copies the newest snapshot onto `silver.questions` only if it is newer than `metrics_as_of`. These are the 7,285 updates in the first Silver run.

Show it: after a run, query `silver.questions` (timestamps, no names) and `silver.dq_quarantine`.

3.7 MERGE and idempotency

Rubric: Use `MERGE INTO` to insert new records and update existing ones. If you run the same input twice, Silver should look exactly the same afterwards, with no duplicate rows.

What we did: every Silver table is written with `MERGE INTO`. The statements are built by `delta_utils.merge_sql()` and run by `delta_utils.run_merge()`, which reads the inserted/updated/deleted counts from the Delta transaction log (`DESCRIBE HISTORY ... operationMetrics`).

The main statement, for `silver.questions` (`questions_merge_sql()`), shortened:

```
MERGE INTO workspace.so_silver.questions AS t
USING src_questions AS s
ON t.question_id = s.question_id
WHEN MATCHED AND (s.source_extracted_at > t.source_extracted_at
                  AND s.record_hash <> t.record_hash) THEN UPDATE SET
    t.title = s.title, ...,
    t.score = CASE WHEN s.source_extracted_at >= t.metrics_as_of THEN s.score ELSE t.score END,
    t.metrics_as_of = GREATEST(s.source_extracted_at, t.metrics_as_of),
    ...
WHEN NOT MATCHED THEN INSERT (...) VALUES (...)
```

Explain the update condition. This is the most likely deep question. A row is updated only if the source is **newer** *and* **different**:

- `s.source_extracted_at > t.source_extracted_at` (newer): every Silver row remembers the extraction time of the version it holds. If you replay an **old** batch after a newer one, the old data cannot overwrite the newer data. Out-of-order backfills are safe.
- `s.record_hash <> t.record_hash` (different): `record_hash` is a SHA-256 of the business columns. If the content did not change, there is no update. That avoids pointless rewrites and keeps `load_timestamp` stable.
- Same input twice means same `source_extracted_at` and same `record_hash`, so the condition is false for every row: **0 updates**. Every key already exists: **0 inserts**. Silver is identical.

Other details worth knowing:

- `first_loaded_at` is never in the `UPDATE SET`, so it keeps the original insert time.
- Metrics (`score`, `view_count`, ...) use `CASE WHEN ... >= t.metrics_as_of`, because a snapshot may already have written fresher metrics than the question record carries.
- A question that was soft-deleted but shows up again in a newer batch is un-deleted.
- `silver.question_tags` uses all three `MERGE` clauses: `WHEN MATCHED ... DELETE` (tag removed by an edit), `WHEN MATCHED ... UPDATE`, `WHEN NOT MATCHED ... INSERT`.
- `silver.users` merges with `LEAST(first_seen_at)` and `GREATEST(last_seen_at)`, so the order in which batches are processed does not matter.

The proof: evidence task `t08` re-merged every batch (90,750 source rows): inserted 0, updated 0 in every Silver table. Section 5 of notebook 03 shows 0 duplicate keys in all six Silver tables.

Show it live: demo steps 4 to 6 in section 5.2. Section 4 of notebook 03 prints the real `MERGE` statement and the `DESCRIBE HISTORY` rows for every `MERGE` on `silver.questions`.

3.8 Parameters and backfills

Rubric: Both steps should take a parameter such as a date, batch ID or folder path. You should be able to re-run any past period without editing the code.

What we did: both notebooks have widgets, and the job, the CLI and the tests pass the same parameters into `run_raw_to_bronze()` and `run_bronze_to_silver()`.

Parameter	Raw-to-Bronze	Bronze-to-Silver	Example
`mode`	`incremental` / `backfill`	`incremental` / `backfill`	`backfill`
`batch_ids`	yes	yes	`incr_20261007T160033Z` (comma-separated for several)
`start_date`, `end_date`	yes	yes	`2026-09-25`, `2026-10-08` (inclusive, extraction date)
`load_type`	`all` / `full` / `incremental`	no	`full`
`landing_path`	yes	no	`/Volumes/workspace/so_raw/landing/incremental/incr_20261007T160033Z`
`drift_mode`	`evolve` / `rescue`	no	`evolve`

How the modes differ:

- **incremental:** Raw-to-Bronze loads every landing batch the registry does not show as loaded. Bronze-to-Silver processes every batch whose Bronze load is newer than its last Silver run (`silver_processed_at IS NULL OR silver_processed_at < bronze_loaded_at OR silver_status <> 'SUCCESS'`).
- **backfill:** re-processes exactly the batches you select, **even if they were done before**. A backfill with no scope is refused with an error (`backfill needs batch_ids, start_date/end_date or landing_path`), so nobody re-processes everything by accident.
- Backfills are safe because Bronze replaces the batch's own rows and Silver merges idempotently.
- Nothing assumes "today". `batches.py` discovers folders under the landing root and filters them by id or date (`select_batches()`). A batch id that does not exist is logged as a `FAILED` step.

Proof: evidence tasks `t07` and `t08` (`mode=backfill`, `start_date=2026-09-25`, `end_date=2026-10-08`). Section 6 of notebook 03.

Likely question: *"How would you backfill a period you never extracted?"* Run `ingestion/full_load.py --from YYYY-MM-DD --to YYYY-MM-DD` (or notebook `99_ingest_api_to_landing`). It writes a new batch folder. Then a normal incremental run picks it up automatically.

3.9 Schema drift

Rubric: If the source adds a column or changes a data type, the pipeline must not crash. Either evolve the schema (`mergeSchema`) or quarantine the bad records and let the rest of the batch finish.

We do both. `drift.py` builds a Python UDF (`make_contract_check`) that compares every raw line with the contract, recursively, including nested structs and arrays. For each line it returns:

- `is_json`: is the line a JSON object at all?
- `unexpected`: fields that are not in the contract (top-level like `ai_assisted`, or nested like `owner.badge_counts`)
- `type_errors`: fields whose value does not fit the type, e.g. `score` = `"five"` for an integer, or a number too big for `int`

`bronze.load_entity_file` then gives every record a `_reason`:

Situation	Reason	Action
Line is not JSON	`malformed_json`	Quarantine
A field has the wrong type	`type_mismatch`	Quarantine (with the field, expected and actual type)
Primary key missing	`missing_primary_key`	Quarantine
New **top-level** field, `drift_mode=evolve`	(valid)	Add the column to Bronze with `mergeSchema`
New **nested** field, or `drift_mode=rescue`	(valid)	Keep it as JSON in `_rescued_data`

Every drift case is also written to `ops.schema_drift_events` (type, column path, record count, example value, action taken). The batch finishes with status SUCCESS, and the log message lists what happened.

Why nested fields are rescued, not evolved: changing the type of a nested struct would change the shape of the `owner` column for every existing row. Adding a top-level column is safe (old rows get null). Keeping nested extras as JSON loses nothing.

Why the UDF is needed: with an explicit schema Spark quietly ignores unknown fields and may turn type mismatches into nulls. Our checker makes drift visible instead of silently losing data.

The Unity Catalog detail: a `mergeSchema` write can drop the table's Unity Catalog primary key constraint. After every schema-evolving write, `ddl.ensure_primary_key()` checks `information_schema.table_constraints` and adds the constraint back. The log message then says "primary key constraint re-applied".

The demo batch (`demo.make_drift_batch`, notebook `04_schema_drift_demo`):

Line	Problem	Result in the evidence run
1	new column `ai_assisted`, new nested `owner.badge_counts`	loaded; `ai_assisted` added by `mergeSchema`; `owner.badge_counts` in `_rescued_data`
2	`score` is `"five"`	quarantined, `type_mismatch`: `expected int, got str: "five"`
3	truncated JSON	quarantined, `malformed_json`
4	no `question_id`	quarantined, `missing_primary_key`
5	valid	loaded

Result: read 5, inserted 2, quarantined 3, status **SUCCESS**. The next Silver run picked up the 2 valid rows.

Show it live: run `04_schema_drift_demo` (demo step 7). It displays the log row, the drift events, the quarantined lines and the evolved column.

3.10 Execution logs

Rubric: A log table such as `pipeline_execution_logs`. Every run writes a row, for full and incremental loads, in both steps. Each row has the layer, the file or parameter processed, start and end time, status, and rows inserted and updated.

What we did: `audit.py` has `AuditLogger` with a context manager `step()`. Every unit of work runs inside it:

```
with audit.step(LAYER, entity, path, target_table, batch_id=..., load_type=...) as rec:
    load_entity_file(...)        # sets rec.rows_inserted, rec.rows_quarantined, ...
```

- On success the status is `SUCCESS`. If the code raises, the status is `FAILED` and the error message is stored. The row is written in a `finally` block, so **failures are logged too**, with their times.
- Raw-to-Bronze writes **one row per file**. Bronze-to-Silver writes **one row per target table step**.
- A batch already loaded in incremental mode gets a `SKIPPED` row, so even a run with nothing to do leaves a trace.
- After all steps, if anything failed, the run raises an error so the notebook or job shows as failed. The log rows are already written by then.

Columns of `so_ops.pipeline_execution_logs`, mapped to the rubric:

Rubric asks for	Column(s)
Layer	`layer` (`Raw-to-Bronze` / `Bronze-to-Silver`)
File or parameter processed	`source_parameter` (file path, or batch list and date range), `batch_id`, `entity`, `target_table`
Full or incremental	`load_type` (`full` / `incremental` / `mixed`), `run_mode` (`incremental` / `backfill`)
Start and end time	`start_time`, `end_time`, `duration_seconds`
Status	`status` (SUCCESS / `FAILED` / `SKIPPED`), `message`
Rows inserted and updated	`rows_inserted`, `rows_updated`, plus `rows_read`, `rows_deleted`, `rows_quarantined`
Grouping	`run_id` shared by all rows of one invocation, `log_id` per row

Two more Ops tables support it: `ops.batch_registry` (which batches are in Bronze and Silver, and their status) and `ops.schema_drift_events`.

Show it: the last cell of notebooks 01, 02 and 04 displays the log rows of that run. Section 8 of notebook 03 shows the whole table. Demo step 8 creates a `FAILED` row on purpose.

3.11 Code and run guide

Rubric: Readable code, and a README that explains how to run a normal incremental load and a backfill for an older period.

What we did: the README has an "Execution guide: incremental load vs backfill" section with the parameter table and examples for notebooks, the job and the CLI. Each notebook starts with a table explaining its widgets for an incremental run and a backfill. Each module starts with a docstring explaining what it does.

3.12 Proof that it works

Rubric: saved notebook outputs or a short video showing the explicit schemas, a `MERGE` re-run with no duplicates, a backfill, schema change handling and the log table.

What we did: `evidence/` holds all 10 tasks of job run `814281740296490` as Jupyter notebooks with their outputs. GitHub renders them directly. `evidence/README.md` maps every rubric item to a notebook and section.

Proof asked for	Where
Explicit schemas	`t09` section 1
`MERGE` re-run, no duplicates	`t08` (0 inserted, 0 updated) and `t09` section 5 (0 duplicates)
A backfill	`t07`, `t08`, `t09` section 6
Schema change handled	`t05`, `t09` section 7
Log table	last cell of every task, `t09` section 8

4. Code tour, file by file

You should be able to open any of these files and say what it does in one or two sentences.

`src/so_lakehouse/` (the pipeline)

File	What it does	Key names to know
`schemas.py`	Every schema in the project as `StructType`, with comments. Also the `TABLES` registry: layer, name, schema, primary key, description for all 15 tables.	`col()`, `QUESTION_RAW`, `OWNER`, `RAW_CONTRACTS`, `BRONZE_METADATA`, `bronze_schema()`, `SILVER_QUESTIONS`, `EXECUTION_LOGS`, `TABLES`
`config.py`	`PipelineConfig`: catalog, schema names, landing path, drift mode, tracked tags, PII salt. Turns `silver.questions` into `workspace.so_silver.questions`. Works locally without a catalog.	`for_databricks()`, `for_local()`, `table()`, `salt()`
`ddl.py`	Creates the schemas, the landing volume and every table from `TABLES` (`CREATE TABLE IF NOT EXISTS`, so it is safe to re-run). Adds Unity Catalog primary keys.	`setup()`, `create_table_sql()`, `ensure_primary_key()`
`batches.py`	Finds batch folders in the landing volume, reads `_manifest.json`, filters by batch id or date range. A broken manifest marks only that batch as bad.	`Batch`, `discover_batches()`, `select_batches()`, `read_batch()`
`drift.py`	The contract checker UDF that finds unknown fields, type errors and malformed lines.	`make_contract_check()`, `_check()`, `sanitize_column()`
`bronze.py`	Raw-to-Bronze: select batches, parse each file with its contract, route bad records to quarantine, evolve the schema, write with `replaceWhere`, record drift and registry.	`run_raw_to_bronze()`, `load_entity_file()`, `record_drift()`, `upsert_registry_bronze()`
`silver.py`	Bronze-to-Silver: de-duplicate, cast, remove PII, DQ rules, `MERGE` into every Silver table, update the registry.	`run_bronze_to_silver()`, `STEPS`, `transform_questions()`, `latest_per_key()`, `conform()`, `user_key()`, `split_dq()`, `questions_merge_sql()`
`delta_utils.py`	Builds `MERGE` statements and reads their row counts from `DESCRIBE HISTORY`. Uses only SQL and DataFrame APIs, because serverless runs on Spark Connect (no `SparkContext` or RDDs).	`merge_sql()`, `run_merge()`, `table_exists()`
`audit.py`	Writes one log row per step, including failures.	`AuditLogger`, `step()`, `StepRecord`
`demo.py`	Writes the deliberately broken drift batch.	`make_drift_batch()`, `DRIFT_BATCH_ID`
`cli.py`	Command line entry point for local runs: `setup`, `raw-to-bronze`, `bronze-to-silver`, `all`.	
`spark_session.py`	Local Spark session with Delta, for tests and local runs.	

`notebooks/`

Notebook	Widgets	What it does
`_bootstrap`	none	Adds `src/` to `sys.path`, reads widgets (`widget()`), gets the salt from the secret scope (`get_pii_salt()`), builds the config. Every other notebook runs it with `%run ./_bootstrap`.
`00_setup`	`catalog`, `reset`, `seed_demo_landing`	Creates schemas, volume and tables. Copies `data/demo_landing` into the volume. `reset=true` drops all tables first.
`01_raw_to_bronze`	`catalog`, `mode`, `load_type`, `batch_ids`, `start_date`, `end_date`, `landing_path`, `drift_mode`	Calls `run_raw_to_bronze`, then displays this run's log rows.
`02_bronze_to_silver`	`catalog`, `mode`, `batch_ids`, `start_date`, `end_date`, `pii_salt`	Calls `run_bronze_to_silver`, then displays this run's log rows.
`03_audit_and_validation`	`catalog`	The evidence notebook: 8 sections matching the rubric's proof list. Read-only.
`04_schema_drift_demo`	`catalog`, `drift_mode`	Writes the broken batch and loads it as a backfill with `landing_path`.
`99_ingest_api_to_landing`	`load_type`, `since`, `from_date`, `to_date`	Calls the ingestion scripts against the API. Needs internet access from the workspace.

`ingestion/`

File	What it does
`so_api.py`	Stack Exchange API client: paging, backoff, quota handling, optional API key
`full_load.py`	Questions created in a date window, plus their answers, written as one `full/<batch_id>/` folder
`incremental_load.py`	Questions active since a timestamp, plus answers, deletion tombstones and metric snapshots, written as one `incremental/<batch_id>/` folder
`common.py`	Writing JSONL and `_manifest.json`

Other folders

- `tests/test_pipeline.py`: 14 tests on local Spark + Delta covering every requirement, including `test_rerun_incremental_is_a_no_op`, `test_full_reprocess_of_same_raw_data_changes_nothing_in_silver`, `test_schema_drift_is_handled_without_crashing` and `test_broken_batch_is_logged_as_failed_and_others_continue`.
- `jobs/so_lakehouse_pipeline.job.json`: a two-task job (`raw_to_bronze` then `bronze_to_silver`) with shared job parameters, one retry per task and a paused daily schedule at 06:00 UTC.
- `scripts/`: data dictionary generator, the Databricks run exporter, the PDF builder, a local demo runner.

5. Running it in Databricks during the evaluation

5.1 The day before: checklist

Do this the day before, not in the room.

1. **Log in** to the workspace (`https://dbc-f67bb652-2597.cloud.databricks.com`) with the account that owns it.
2. **Find the code: Workspace -> Users -> your email ->** `stackoverflow-lakehouse`. You should see `notebooks/`, `src/`, `config/`, `data/`. This folder was uploaded from the repo. If you changed code on GitHub since, upload it again (see 5.5).
3. **Check the tables: Catalog -> workspace**. You should see the schemas `so_raw`, `so_bronze`, `so_silver`, `so_ops`, and inside `so_raw` the volume `landing` with folders `full/` and `incremental/`.
4. **Check the secret.** With the Databricks CLI: `databricks secrets list-secrets so_lakehouse` should list `pii_salt`. If the CLI is not available, just run notebook 02 once: if the secret is missing it stops with "Create the secret so_lakehouse/pii_salt". As a last resort, type any long string into the `pii_salt` widget of notebook 02. Use a **different** salt only on a fresh reset, because changing the salt changes every `user_key`.
5. **Do a dry run of the demo script below** so serverless compute is warm and you know the timings. On serverless the first cell can take about a minute to start. The evidence run took about 3 minutes for the full Raw-to-Bronze and

2 to 3 minutes for each Silver run.

6. **Open these tabs** before the evaluation starts: the GitHub repo, `evidence/README.md` on GitHub, the Databricks workspace folder, Catalog Explorer, and a SQL editor (**SQL Editor** in the left bar) or an empty scratch notebook.

5.2 The live demo script (about 10 minutes)

This order covers every rubric item. It **does not reset anything**, so the existing data stays intact. Each step says what to set, what to click and what you should see.

How to set widgets: open the notebook. The widgets appear as a bar at the top. Change the values there, then click **Run all** at the top. If the bar is hidden, the notebook has not been run yet: run the first cell once and the widgets appear.

Step 1: Code and repo (1 minute). Show the GitHub repo layout, `src/so_lakehouse/schemas.py` (the `StructType` contracts), and the README data model section.

Step 2: Tables in Unity Catalog (1 minute). Catalog -> workspace -> so_silver -> questions: columns with types and comments, and the primary key. Then **so_bronze -> questions -> Sample data**: point at `_batch_id`, `_source_file`, `load_timestamp`.

Step 3: A normal incremental run that finds nothing new.

- Open `notebooks/01_raw_to_bronze`. Widgets: `mode = incremental`, everything else empty (defaults). **Run all**.
- Expected: every batch is logged as `SKIPPED` with "already loaded; use mode=backfill to reload". The last cell shows those log rows.
- Say: "The batch registry knows every batch is already in Bronze, so the incremental run does nothing, and it still writes a log row."

Step 4: Count Silver before the re-run. In the SQL editor, run the "Silver counts and duplicates" query from 5.4. Note the numbers.

Step 5: Backfill one batch through both layers.

- `01_raw_to_bronze`: `mode = backfill`, `batch_ids = incr_20261007T160033Z`, other fields empty. **Run all**.
  - Expected: `SUCCESS` rows for `questions`, `answers`, `deleted_questions`, `question_snapshots`. The message says "replaced 51 rows from a previous load of this batch" for questions (84 for answers). `rows_inserted` equals `rows_deleted`: the batch was replaced, not duplicated.
- `02_bronze_to_silver`: `mode = backfill`, `batch_ids = incr_20261007T160033Z`. **Run all**.
  - Expected: 9 `SUCCESS` rows, all with `rows_inserted = 0` and `rows_updated = 0`.
  - Say: "Same input, so every row has the same `source_extracted_at` and the same `record_hash`. The `MERGE` condition is false for every row: nothing inserted, nothing updated."

Step 6: Prove Silver did not change. Run the "Silver counts and duplicates" query again. Same row counts as in step 4, and 0 duplicates everywhere.

You can also show a date-range backfill: the same two notebooks with `mode = backfill`, `start_date = 2026-09-25`, `end_date = 2026-09-25`, `batch_ids` empty. That selects the two batches extracted on 25 September (`full_2025-04-01_2025-07-01` and `incr_20260925T155859Z`) and leaves the others alone; the first output line says "2 batch(es) selected". It takes a few minutes longer than the single small batch, so only do it if there is time.

Step 7: Schema drift.

- Open `notebooks/04_schema_drift_demo`, `drift_mode = evolve`, **Run all**.
- Expected (four outputs):
  1. Log row: `SUCCESS`, `rows_read = 5`, `rows_inserted = 2`, `rows_quarantined = 3`.
  2. Drift events: `ai_assisted` evolved, `owner.badge_counts` rescued, `score` type mismatch quarantined, missing `question_id` quarantined, malformed JSON quarantined.
  3. Quarantine: the three bad raw lines with their reason and error detail.
  4. Bronze: questions `990000001` (with `ai_assisted = true` and `_rescued_data`) and `990000004`.
- Because this batch was loaded before, the message says "replaced 2 rows" and the column `ai_assisted` already exists, so there is no "schema evolved" note this time. Point to the evidence notebook `t05` for the first run, where it says "schema evolved: added ai_assisted; primary key constraint re-applied".
- Say: "A new column, a type change, broken JSON and a missing key in one file. The batch did not crash. Good rows were loaded, bad rows went to quarantine with the reason, and every event is recorded."

Step 8: A failure is logged too.

- `01_raw_to_bronze`: `mode = backfill`, `batch_ids = incr_does_not_exist`. **Run all**.
- Expected: the run cell ends with a red `RuntimeError: 1 Raw-to-Bronze step(s) failed; see ... run_id=...`. **That is intended.** Then run the "Latest log rows" query: the newest row has `status = FAILED` and `message = FileNotFoundError: batch incr_does_not_exist not found under /Volumes/...`.
- Say: "The log is written in a `finally` block, so failures are recorded with start time, end time and the error, and then the run reports failure to the job."

Step 9: The log table. Run the "Log summary per run" query, then the "Latest log rows" query. Point at `layer`, `source_parameter`, `start_time`, `end_time`, `status`, `rows_inserted`, `rows_updated`, and at both `full` and `incremental` in `load_type`.

Step 10 (if asked for everything at once): open `03_audit_and_validation` and **Run all** (about 2 minutes). It walks through all 8 proofs in order.

5.3 Using the job instead of notebooks

The evidence was made with one-off job runs, so there may be no saved job in the workspace. To create one:

1. **Workflows (Jobs & Pipelines) -> Create job.**
2. Task 1: name `raw_to_bronze`, type **Notebook**, path `.../stackoverflow-lakehouse/notebooks/01_raw_to_bronze`, compute **Serverless**.
3. Task 2: name `bronze_to_silver`, notebook `02_bronze_to_silver`, **Depends on** `raw_to_bronze`.
4. **Job parameters:** add `catalog = workspace`, `mode = incremental`, `batch_ids`, `start_date`, `end_date` (empty values are fine). Job parameters are passed to every notebook task and fill the widgets with the same names.
5. **Run now** does a standard incremental load. **Run now with different parameters** with `mode = backfill` and `start_date`/`end_date` or `batch_ids` does a backfill.

Or with the CLI, after replacing `<your-email>` in the notebook paths of `jobs/so_lakehouse_pipeline.job.json`:

```
databricks jobs create --json @jobs/so_lakehouse_pipeline.job.json
databricks jobs run-now <job_id>
databricks jobs run-now <job_id> --json \
  '{"job_parameters": {"mode": "backfill", "start_date": "2026-09-25", "end_date": "2026-09-25"}}'
```

5.4 SQL snippets to paste during the demo

Paste these into the SQL editor (catalog `workspace`) or a notebook cell starting with `%sql`.

Silver counts and duplicates (run before and after the re-run):

```
SELECT 'questions' AS tbl, count(*) AS rows,
       count(*) - count(DISTINCT question_id) AS duplicates
  FROM workspace.so_silver.questions
UNION ALL
SELECT 'answers', count(*), count(*) - count(DISTINCT answer_id)
  FROM workspace.so_silver.answers
UNION ALL
SELECT 'question_tags', count(*), count(*) - count(DISTINCT question_id, tag)
  FROM workspace.so_silver.question_tags
UNION ALL
SELECT 'users', count(*), count(*) - count(DISTINCT user_key)
  FROM workspace.so_silver.users
UNION ALL
SELECT 'question_snapshots', count(*), count(*) - count(DISTINCT question_id, snapshot_at)
  FROM workspace.so_silver.question_snapshots
UNION ALL
SELECT 'post_text_masked', count(*), count(*) - count(DISTINCT post_type, post_id)
  FROM workspace.so_silver.post_text_masked;
```

Latest log rows:

```
SELECT layer, run_mode, load_type, entity, batch_id, source_parameter, start_time, end_time,
       status, rows_read, rows_inserted, rows_updated, rows_deleted, rows_quarantined, message
FROM workspace.so_ops.pipeline_execution_logs
ORDER BY start_time DESC
LIMIT 25;
```

Log summary per run:

```
SELECT run_id, layer, run_mode, min(start_time) AS started, max(end_time) AS finished,
       count(*) AS steps, sum(rows_inserted) AS inserted, sum(rows_updated) AS updated,
       sum(rows_quarantined) AS quarantined, concat_ws(',', collect_set(status)) AS statuses
FROM workspace.so_ops.pipeline_execution_logs
GROUP BY run_id, layer, run_mode
ORDER BY started;
```

Bronze lineage (where each record came from):

```
SELECT _batch_id, _load_type, _batch_date, _extracted_at, _source_file,
       count(*) AS rows, max(load_timestamp) AS loaded
FROM workspace.so_bronze.questions
GROUP BY ALL
ORDER BY _extracted_at;
```

One question that appears in several Bronze batches, and its single Silver row:

```
-- pick a question that is in more than one batch
SELECT question_id, count(*) AS batches
FROM workspace.so_bronze.questions
GROUP BY question_id HAVING count(*) > 1
LIMIT 1;

-- then put that id into both queries below
SELECT question_id, _batch_id, score, view_count, last_activity_date, load_timestamp
FROM workspace.so_bronze.questions WHERE question_id = <id>;

SELECT question_id, source_batch_id, score, view_count, last_activity_at,
       first_loaded_at, load_timestamp
FROM workspace.so_silver.questions WHERE question_id = <id>;
```

What each MERGE did (Delta transaction log):

```
DESCRIBE HISTORY workspace.so_silver.questions;
```

Look at the rows with `operation = MERGE`: the `operationMetrics` column holds `numSourceRows`, `numTargetRowsInserted` and `numTargetRowsUpdated`. Section 4 of notebook 03 shows the same thing as a clean table.

Drift, quarantine, registry, PII check:

```
SELECT _batch_id, entity, drift_type, column_path, record_count, action_taken, example_value
FROM workspace.so_ops.schema_drift_events;

SELECT entity, _batch_id, reason, error_detail, raw_record
FROM workspace.so_bronze.quarantine_records;

SELECT batch_id, load_type, batch_date, bronze_status, bronze_loaded_at,
       silver_status, silver_processed_at
FROM workspace.so_ops.batch_registry ORDER BY extracted_at;

-- no names in Silver, only the hashed user key
SELECT question_id, title, created_at, owner_user_key, owner_reputation
FROM workspace.so_silver.questions LIMIT 5;

SELECT post_type, post_id, masked_token_count, body_masked
FROM workspace.so_silver.post_text_masked WHERE masked_token_count > 0 LIMIT 5;
```

5.5 If you must start from scratch

Use this if the workspace was wiped, or the examiner asks to see a clean run. It takes about 15 minutes.

1. **Get the code into the workspace**, either way:
   - **Git folder: Workspace -> Create -> Git folder**, paste `https://github.com/Suf-7/stackoverflow-lakehouse`. The repo is private, so first add a GitHub token under **Settings -> Linked accounts**.
   - **Upload with the CLI** from a laptop with the repo cloned: `databricks workspace import-dir . /Workspace/Users/<your-email>/stackoverflow-lakehouse --overwrite`. Python files with the `# Databricks notebook source` header become notebooks; the rest become plain files.
2. **Secret** (once per workspace):

```
databricks secrets create-scope so_lakehouse
databricks secrets put-secret so_lakehouse pii_salt --string-value "<long random string>"
```

3. Run the notebooks in this order. This is exactly the evidence run:

#	Notebook	Widgets	Expected
1	`00_setup`	`reset = true`, `seed_demo_landing = true`	15 tables created; demo batches copied to the volume
2	`01_raw_to_bronze`	`mode = incremental`	16 files SUCCESS, 24,764 rows (plus the drift batch if its folder is still in the volume)
3	`02_bronze_to_silver`	`mode = incremental`	9 steps SUCCESS
4	`01_raw_to_bronze`	`mode = incremental` (again)	every batch `SKIPPED`
5	`02_bronze_to_silver`	`mode = incremental` (again)	one `SKIPPED` row: "no matching Bronze batches to process"
6	`04_schema_drift_demo`	`drift_mode = evolve`	2 loaded, 3 quarantined, `SUCCESS`
7	`02_bronze_to_silver`	`mode = incremental`	the 2 drift rows reach Silver
8	`01_raw_to_bronze`	`mode = backfill`, `start_date = 2026-09-25`, `end_date = 2026-10-08`	every batch replaced, nothing duplicated
9	`02_bronze_to_silver`	`mode = backfill`, same dates	0 inserted, 0 updated everywhere
10	`03_audit_and_validation`	none	all 8 proofs

`reset = true` drops every table. It does **not** delete the files in the volume. The drift demo folder `incr_20261008T000000Z` stays there, so after a reset step 2 also loads it.

5.6 Troubleshooting

Symptom	Cause	Fix
`ModuleNotFoundError: so_lakehouse`	Notebook opened outside the repo folder, or `src/` was not uploaded	Open the notebook from `stackoverflow-lakehouse/notebooks/`; check `src/so_lakehouse/` exists next to it
"Create the secret so_lakehouse/pii_salt"	Secret missing	Create it (5.5 step 2) or fill the `pii_salt` widget
`TABLE_OR_VIEW_NOT_FOUND`	Setup never ran in this workspace	Run `00_setup`
`backfill needs batch_ids, start_date/end_date or landing_path`	Backfill without a scope	Fill `batch_ids` or the dates. This guard is deliberate.
Every step `SKIPPED`	Incremental mode, nothing new	Expected. Use `mode = backfill` to re-run.
Widgets missing	Notebook has not run yet	Run the first cell once
First cell waits a long time	Serverless compute starting	Wait about a minute
`99_ingest_api_to_landing` fails with a network error	Workspace cannot reach the internet, or API quota used up (300 requests a day without a key)	Do not use it in the demo. The batches are already in the volume.

6. Questions the examiner may ask

Architecture and design

1. **Why Medallion?** Each layer has one job. Bronze is the replayable record of what arrived, Silver is the cleaned truth, Gold (Phase 3) will be the business aggregates. If Silver logic has a bug, we fix the code and re-run from Bronze. We never need to call the API again.
2. **Why Delta Lake?** ACID transactions, `MERGE INTO`, `replaceWhere`, schema evolution, and `DESCRIBE HISTORY` for row counts and time travel. Plain Parquet has none of these.
3. **Why a landing volume instead of reading the API straight into Spark?** It separates extraction from processing. Raw files are kept, so any backfill reads them again without API calls and API quota.
4. **Why are the notebooks so short?** Logic is in the `so_lakehouse` package so it can be tested locally and reused by notebooks, the job and the CLI.

5. **Why serverless-safe code?** Free Edition is serverless and runs on Spark Connect, which has no `SparkContext` or RDD API. `delta_utils.py` uses only SQL and DataFrames, for example it reads `MERGE` counts from `DESCRIBE HISTORY`.

Schemas

6. **Where is the schema defined, and when?** In `schemas.py`, as `StructType` objects, before any read. `from_json` receives the contract.
7. **What does PERMISSIVE mode do?** Instead of failing the job on a bad record, Spark sets fields it cannot parse to null and puts the original text in `_corrupt_record`. We use that plus our checker to route the record to quarantine.
8. **Why are some nested fields `STRING`?** `posted_by_collectives`, `recommendations` and `migrated_from.other_site` are rarely used and have many sub-fields. As `STRING`, Spark keeps their raw JSON text, so nothing is lost and the contract stays small.
9. **Why are Bronze dates still `bigint`?** Bronze stores data as it arrived. Casting to timestamps is a Silver job (`ts()`).

Idempotency and MERGE

10. **What does idempotent mean here?** Running the pipeline again on the same raw data leaves Silver exactly as it was: same rows, same values, no duplicates.
11. **Walk me through your MERGE.** See 3.7: match on the key, update only if newer and different, insert unknown keys.
12. **What if you replay an old batch after a newer one?** The `s.source_extracted_at > t.source_extracted_at` condition stops the old version overwriting the newer one.
13. **Why `record_hash`?** It detects whether content really changed. Without it, re-processing a batch with the same extraction time could still rewrite rows.
14. **Why de-duplicate before MERGE?** `MERGE` fails if two source rows match the same target row. `latest_per_key()` keeps exactly one row per key.
15. **How do you count inserted and updated rows?** From the Delta commit: `DESCRIBE HISTORY` gives `numTargetRowsInserted` and `numTargetRowsUpdated` for the `MERGE` we just ran.
16. **Is Bronze idempotent too?** Yes. `replaceWhere _batch_id = '<id>'` replaces that batch's rows. The quarantine table uses the same trick per entity and batch, and drift events and the registry use `MERGE`.

Backfills

17. **How do you run a backfill?** `mode = backfill` plus `batch_ids`, or `start_date` and `end_date`, or `landing_path`. No code changes.
18. **Which date does the date range use?** The extraction date of the batch (`_batch_date`, from `_manifest.json`).
19. **What stops someone re-processing everything by mistake?** A backfill without a scope raises an error.

Schema drift

20. **What happens if the API adds a column?** Top-level: added to the Bronze table with `mergeSchema` (`drift_mode = evolve`), or kept in `_rescued_data` (`drift_mode = rescue`). Nested: always kept in `_rescued_data`. Either way it is logged in `ops.schema_drift_events`.
21. **What if `score` becomes a string?** The record goes to `bronze.quarantine_records` with `reason = type_mismatch` and the detail `expected int, got str: "five"`. The rest of the file loads.
22. **Can a quarantined record be recovered?** Yes. `raw_record` holds the untouched line. After fixing the contract or the data, it can be loaded again.
23. **Why did you need `ensure_primary_key()`?** On Databricks, a `mergeSchema` write dropped the Unity Catalog primary key of `bronze.questions`. We re-add it after every schema-evolving write.

Logging

24. **What if the pipeline crashes halfway?** The failed step's log row is written in the `finally` block with `status = FAILED` and the error. The batch registry marks the batch `FAILED`, so the next incremental run retries it.
25. **What is a `run_id`?** One id shared by all log rows of one notebook run, so you can group a run's steps together.
26. **Why does Raw-to-Bronze log per file but Silver per table?** That matches the unit of work in each layer: Bronze processes files, Silver processes target tables across a set of batches.

PII and security

27. **Why a salted hash instead of plain SHA-256?** Stack Overflow user ids are public and sequential. Anyone could hash every id and reverse a plain hash. With a secret salt they cannot.

28. **Where is the salt?** In the Databricks secret scope `so_lakehouse`, key `pii_salt`. It never appears in code or output.
29. **Why keep the text at all?** For text analysis in Phase 3. It is in a separate restricted table (`post_text_masked`) with emails, IPs, secrets and connection strings masked.

Testing and cost

30. **How did you test it?** 14 end-to-end tests on local Spark + Delta, then the 10-task job run on Databricks saved in `evidence/`.
31. **How do you stay inside the free tier?** A few MB of data per batch, serverless compute that stops when idle, demo batches committed to the repo so we do not call the API repeatedly, and the daily job schedule left paused.

---

7. Who presents what (suggested split)

Part	Presenter
Pitch, architecture, data source, PII	Member 1
Schemas, Bronze, schema drift (demo steps 1, 2, 3, 7)	Member 1
Silver, MERGE, idempotency, backfill (demo steps 4, 5, 6)	Member 2
Logging and failures (demo steps 8, 9), evidence folder	Member 2

Both members should still be able to answer any question in section 6. Examiners often ask the person who did not present a part.

---

8. Last-minute cheat sheet

- Schemas: `schemas.py`, `StructType`, `from_json` PERMISSIVE, 0 `inferSchema`.
- Casting: `ts()` = `timestamp_seconds`, `conform()` casts to the Silver contract.
- `load_timestamp` in all 15 tables; Silver also has `first_loaded_at`.
- Bronze lineage: `_batch_id`, `_load_type`, `_batch_date`, `_extracted_at`, `_source_file`, `_record_hash`, `_rescued_data`.
- Bronze idempotency: `replaceWhere _batch_id = ...`.
- Silver idempotency: `MERGE`, update when **newer and different** (`source_extracted_at`, `record_hash`), de-duplicate first with `latest_per_key()`.
- Proof: 90,750 rows re-merged, 0 inserted, 0 updated, 0 duplicates.
- Backfill: `mode = backfill` + `batch_ids` / `start_date`, `end_date` / `landing_path`.
- Drift: top-level column evolved with `mergeSchema`, nested kept in `_rescued_data`; type mismatch, malformed JSON, missing key quarantined; batch SUCCESS; `schema_drift_events`.
- Logs: `so_ops.pipeline_execution_logs`, one row per file or table, written in `finally`, `SUCCESS` / `FAILED` / `SKIPPED`.
- Quick demo batch: `incr_20261007T160033Z`. Failure demo: `batch_ids = incr_does_not_exist`.